

template.cpp

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define rep(i, n)
4 #define all(a) begin(a), end(a)
5 using ll = long long;
6 using vi = vector<int>;
7 using vl = vector<ll>;
8 template <class T>
9 bool chmax(T& a, const T& b) {
10     if (a < b) {
11         a = b;
12         return 1;
13     }
14     return 0;
15 }
16 template <class T>
17 bool chmin(T& a, const T& b) {
18     if (b < a) {
19         a = b;
20         return 1;
21     }
22     return 0;
23 }
24
25 void solve() {
26 }
27
28 signed main() {
29     cin.tie(0);
30     ios::sync_with_stdio(0);
31     cout << fixed << setprecision(20);
32     int TT;
33     TT = 1; //cin >> TT;
34     solve();
35 }

```

dsu.cpp

```

1 struct dsu {
2     vector<int> p;
3     dsu(int n) : p(n, -1) {}
4     int leader(int x) {
5         return p[x] < 0 ? x
6             : p[x] = leader(p[x]);
7     }
8     int merge(int a, int b) {
9         a = leader(a);
10        b = leader(b);
11        if (a == b)
12            return a;
13        if (p[a] > p[b])
14            swap(a, b);
15        p[a] += p[b];
16        p[b] = a;
17        return a;
18    }
19    bool same(int a, int b) {
20        return leader(a) == leader(b);
21    }
22    int size(int a) { return -p[leader(a)]; }
23 };

```

segtree.cpp

```

1 template <class S, S op(S, S), S e()>
2 struct segtree {
3     int n, sz;
4     vector<S> d;

```

segtree.cpp (cont.)

```

5     segtree(int m = 0) : n(1), sz(m) {
6         while (n < m)
7             n *= 2;
8         d.assign(2 * n, e());
9     }
10    segtree(vector<S> a) : segtree(a.size()) {
11        copy(a.begin(), a.end(), d.begin() + n);
12        for (int i = n - 1; i; i--)
13            d[i] = op(d[2 * i], d[2 * i + 1]);
14    }
15    void set(int p, S x) {
16        for (d[p += n] = x; p /= 2;)
17            d[p] = op(d[2 * p], d[2 * p + 1]);
18    }
19    S get(int p) { return d[p + n]; }
20    S prod(int l, int r) {
21        S x = e(), y = e();
22        for (l += n, r += n; l < r;
23             l /= 2, r /= 2) {
24            if (l & 1)
25                x = op(x, d[l++]);
26            if (r & 1)
27                y = op(d[--r], y);
28        }
29        return op(x, y);
30    }
31    S all_prod() { return d[1]; }
32
33    template <class G>
34    int max_right(int l, G g) {
35        assert(g(e()));
36        S x = e();
37        int r = mr(l, g, x, 1, 0, n);
38        return r < 0 ? sz : r;
39    }
40    template <class G>
41    int mr(int a, G& g, S& x, int k, int l,
42           int r) {
43        if (r <= a || sz <= l)
44            return -1;
45        if (a <= l && r <= sz) {
46            S y = op(x, d[k]);
47            if (g(y)) {
48                x = y;
49                return -1;
50            }
51            if (r - l == 1)
52                return l;
53        }
54        int m = (l + r) / 2,
55            q = mr(a, g, x, 2 * k, l, m);
56        return q < 0
57            ? mr(a, g, x, 2 * k + 1, m, r)
58            : q;
59    }
60    template <class G>
61    int min_left(int r, G g) {
62        assert(g(e()));
63        S x = e();
64        int l = ml(r, g, x, 1, 0, n);
65        return l < 0 ? 0 : l;
66    }
67    template <class G>
68    int ml(int b, G& g, S& x, int k, int l,
69           int r) {
70        if (b <= l)
71            return -1;

```

segtree.cpp (cont.)

```

72     if (r <= b) {
73         S y = op(d[k], x);
74         if (g(y)) {
75             x = y;
76             return -1;
77         }
78         if (r - l == 1)
79             return r;
80     }
81     int m = (l + r) / 2;
82     q = ml(b, g, x, 2 * k + 1, m, r);
83     return q < 0 ? ml(b, g, x, 2 * k, l, m)
84           : q;
85 }
86 };

```

lazy_segtree.cpp

```

1  template <class S, S op(S, S), S e(),
2  class F, S mapping(F, S),
3  F composition(F, F), F id()>
4  struct lazy_segtree {
5      int n, sz;
6      vector<S> d;
7      vector<F> lz;
8      lazy_segtree(int m = 0) : n(1), sz(m) {
9          while (n < m)
10             n *= 2;
11         d.assign(2 * n, e());
12         lz.assign(2 * n, id());
13     }
14     lazy_segtree(vector<S> a)
15         : lazy_segtree(a.size()) {
16         copy(a.begin(), a.end(), d.begin() + n);
17         for (int i = n - 1; i--; i--)
18             d[i] = op(d[2 * i], d[2 * i + 1]);
19     }
20     void all(int k, F f) {
21         d[k] = mapping(f, d[k]);
22         lz[k] = composition(f, lz[k]);
23     }
24     void push(int k) {
25         all(2 * k, lz[k]);
26         all(2 * k + 1, lz[k]);
27         lz[k] = id();
28     }
29     void set(int p, S x) {
30         set(p, x, 1, 0, n);
31     }
32     void set(
33         int p, S x, int k, int l, int r) {
34         if (r - l == 1) {
35             d[k] = x;
36             lz[k] = id();
37             return;
38         }
39         push(k);
40         int m = (l + r) / 2;
41         p < m ? set(p, x, 2 * k, l, m)
42              : set(p, x, 2 * k + 1, m, r);
43         d[k] = op(d[2 * k], d[2 * k + 1]);
44     }
45     S get(int p) { return prod(p, p + 1); }
46     S prod(int a, int b) {
47         return prod(a, b, 1, 0, n);
48     }
49     S prod(

```

lazy_segtree.cpp (cont.)

```

50     int a, int b, int k, int l, int r) {
51         if (r <= a || b <= l)
52             return e();
53         if (a <= l && r <= b)
54             return d[k];
55         push(k);
56         int m = (l + r) / 2;
57         return op(prod(a, b, 2 * k, l, m),
58                 prod(a, b, 2 * k + 1, m, r));
59     }
60     S all_prod() { return d[1]; }
61     void apply(int p, F f) {
62         apply(p, p + 1, f);
63     }
64     void apply(int a, int b, F f) {
65         apply(a, b, f, 1, 0, n);
66     }
67     void apply(int a, int b, F f, int k,
68               int l, int r) {
69         if (r <= a || b <= l)
70             return;
71         if (a <= l && r <= b) {
72             all(k, f);
73             return;
74         }
75         push(k);
76         int m = (l + r) / 2;
77         apply(a, b, f, 2 * k, l, m);
78         apply(a, b, f, 2 * k + 1, m, r);
79         d[k] = op(d[2 * k], d[2 * k + 1]);
80     }
81
82     template <class G>
83     int max_right(int l, G g) {
84         assert(g(e()));
85         S x = e();
86         int r = mr(l, g, x, 1, 0, n);
87         return r < 0 ? sz : r;
88     }
89     template <class G>
90     int mr(int a, G& g, S& x, int k, int l,
91           int r) {
92         if (r <= a || sz <= l)
93             return -1;
94         if (a <= l && r <= sz) {
95             S y = op(x, d[k]);
96             if (g(y)) {
97                 x = y;
98                 return -1;
99             }
100             if (r - l == 1)
101                 return l;
102         }
103         push(k);
104         int m = (l + r) / 2;
105         int q = mr(a, g, x, 2 * k, l, m);
106         return q < 0
107             ? mr(a, g, x, 2 * k + 1, m, r)
108             : q;
109     }
110     template <class G>
111     int min_left(int r, G g) {
112         assert(g(e()));
113         S x = e();
114         int l = ml(r, g, x, 1, 0, n);
115         return l < 0 ? 0 : l;
116     }

```

lazy_segtree.cpp (cont.)

```

117 template <class G>
118 int ml(int b, G& g, S& x, int k, int l,
119 int r) {
120     if (b <= l)
121         return -1;
122     if (r <= b) {
123         S y = op(d[k], x);
124         if (g(y)) {
125             x = y;
126             return -1;
127         }
128         if (r - l == 1)
129             return r;
130     }
131     push(k);
132     int m = (l + r) / 2;
133     int q = ml(b, g, x, 2 * k + 1, m, r);
134     return q < 0 ? ml(b, g, x, 2 * k, l, m)
135           : q;
136 }
137 };
138
139 template <class S, S op(S, S), S e()>
140 struct segtree {
141     int n, sz;
142     vector<S> d;
143     segtree(int m = 0) : n(1), sz(m) {
144         while (n < m)
145             n *= 2;
146         d.assign(2 * n, e());
147     }
148     segtree(vector<S> a) : segtree(a.size()) {
149         copy(a.begin(), a.end(), d.begin() + n);
150         for (int i = n - 1; i; i--)
151             d[i] = op(d[2 * i], d[2 * i + 1]);
152     }
153     void set(int p, S x) {
154         for (d[p += n] = x; p /= 2;)
155             d[p] = op(d[2 * p], d[2 * p + 1]);
156     }
157     S get(int p) { return d[p + n]; }
158     S prod(int l, int r) {
159         S x = e(), y = e();
160         for (l += n, r += n; l < r;
161              l /= 2, r /= 2) {
162             if (l & 1)
163                 x = op(x, d[l++]);
164             if (r & 1)
165                 y = op(d[--r], y);
166         }
167         return op(x, y);
168     }
169     S all_prod() { return d[1]; }
170
171 template <class G>
172 int max_right(int l, G g) {
173     assert(g(e()));
174     S x = e();
175     int r = mr(l, g, x, 1, 0, n);
176     return r < 0 ? sz : r;
177 }
178 template <class G>
179 int mr(int a, G& g, S& x, int k, int l,
180 int r) {
181     if (r <= a || sz <= l)
182         return -1;

```

lazy_segtree.cpp (cont.)

```

183     if (a <= l && r <= sz) {
184         S y = op(x, d[k]);
185         if (g(y)) {
186             x = y;
187             return -1;
188         }
189         if (r - l == 1)
190             return l;
191     }
192     int m = (l + r) / 2,
193           q = mr(a, g, x, 2 * k, l, m);
194     return q < 0
195           ? mr(a, g, x, 2 * k + 1, m, r)
196           : q;
197 }
198 template <class G>
199 int min_left(int r, G g) {
200     assert(g(e()));
201     S x = e();
202     int l = ml(r, g, x, 1, 0, n);
203     return l < 0 ? 0 : l;
204 }
205 template <class G>
206 int ml(int b, G& g, S& x, int k, int l,
207 int r) {
208     if (b <= l)
209         return -1;
210     if (r <= b) {
211         S y = op(d[k], x);
212         if (g(y)) {
213             x = y;
214             return -1;
215         }
216         if (r - l == 1)
217             return r;
218     }
219     int m = (l + r) / 2,
220           q = ml(b, g, x, 2 * k + 1, m, r);
221     return q < 0 ? ml(b, g, x, 2 * k, l, m)
222           : q;
223 }
224 };

```

binarytrie.cpp

```

1 template <int B = 30>
2 struct
3     binary_trie { //0 以上 2^B 未満の整数を管理す
4         る Binary Trie
5         struct N {
6             int nx[2] = {-1, -1}, cnt = 0;
7         };
8         vector<N> t{N()};
9
10        void add(long long x,
11        int d = 1) { //x の個数を d だけ増減する
12            int v = 0;
13            t[v].cnt += d;
14            for (int i = B - 1; i >= 0; i--) {
15                int b = x >> i & 1;
16                if (t[v].nx[b] < 0)
17                    t[v].nx[b] = t.size(),
18                    t.emplace_back();
19                v = t[v].nx[b];
20                t[v].cnt += d;
21            }
22        }

```

binarytrie.cpp (cont.)

```

23 void insert(
24     long long x) { //x を 1 個追加する
25     add(x, 1);
26 }
27
28 void erase(long long x) { //x を 1 個削除する
29     add(x, -1);
30 }
31
32 long long min_xor(long long
33     x) { //集合内の y について x xor
34     y の最小値を返す
35     int v = 0;
36     long long r = 0;
37     for (int i = B - 1; i >= 0; i--) {
38         int b = x >> i & 1, u = t[v].nx[b];
39         if (u < 0 || !t[u].cnt)
40             b ^= 1, r |= 1LL << i;
41         v = t[v].nx[b];
42     }
43     return r;
44 }
45
46 long long max_xor(long long
47     x) { //集合内の y について x xor
48     y の最大値を返す
49     int v = 0;
50     long long r = 0;
51     for (int i = B - 1; i >= 0; i--) {
52         int b = x >> i & 1,
53             u = t[v].nx[b ^ 1];
54         if (u >= 0 && t[u].cnt)
55             b ^= 1, r |= 1LL << i;
56         v = t[v].nx[b];
57     }
58     return r;
59 }

```

trie.cpp

```

1 template <int K, char B>
2 struct
3     trie { //K 文字種, B を先頭文字とする文字列 Tr
4         ie
5         struct N {
6             int nx[K], end = 0;
7             N() { fill(nx, nx + K, -1); }
8         };
9         vector<N> t{N()};
10
11 void insert(
12     string s) { //文字列 s を追加する
13     int v = 0;
14     for (char c : s) {
15         int x = c -
16             B; //文字 c に対応する子番号
17         連続でない文字を取りたいなら id 関数を作る
18         if (t[v].nx[x] < 0)
19             t[v].nx[x] = t.size(),
20             t.emplace_back();
21         v = t[v].nx[x];
22     }
23     t[v].end++;
24 }

```

trie.cpp (cont.)

```

24 int find(string
25     s) { //文字列 s に対応する頂点番号を返す
26     (存在しなければ-1)
27     int v = 0;
28     for (char c : s) {
29         v = t[v].nx[c - B];
30         if (v < 0)
31             return -1;
32     }
33     return v;
34 }

```

suffix_array.cpp

```

1 template <class T>
2 vector<int> suffix_array(vector<T> s) {
3     int n = s.size();
4     if (!n)
5         return {};
6     int N = n + 1;
7     vector<int> p(N), c(N), q(N), nc(N),
8         cnt(N);
9     p[0] = n;
10    iota(p.begin() + 1, p.end(), 0);
11    sort(p.begin() + 1, p.end(),
12        [&](int i, int j) {
13            return s[i] < s[j];
14        });
15    for (int i = 1; i < N; i++)
16        c[p[i]] = i == 1 ? 1
17            : c[p[i - 1]] +
18            (s[p[i - 1]] != s[p[i]]);
19    for (int k = 1; k < N; k *= 2) {
20        for (int i = 0; i < N; i++)
21            q[i] = (p[i] - k + N) % N;
22        fill(cnt.begin(), cnt.end(), 0);
23        for (int x : q)
24            cnt[c[x]]++;
25        for (int i = 1; i < N; i++)
26            cnt[i] += cnt[i - 1];
27        for (int i = N - 1; i >= 0; i--)
28            p[--cnt[c[q[i]]]] = q[i];
29        nc[p[0]] = 0;
30        for (int i = 1; i < N; i++)
31            nc[p[i]] = nc[p[i - 1]] +
32            (c[p[i - 1]] != c[p[i]] ||
33            c[(p[i - 1] + k) % N] !=
34            c[(p[i] + k) % N]);
35        c.swap(nc);
36    }
37    return {p.begin() + 1, p.end()};
38 }
39 vector<int> suffix_array(string s) {
40     return suffix_array(vector<unsigned char>(
41         s.begin(), s.end()));
42 }
43 vector<int> suffix_array(
44     vector<int> s, int) {
45     return suffix_array<int>(move(s));
46 }

```

lcp_array.cpp

```

1 template <class T>
2 vector<int> lcp_array(
3     const T& s, const vector<int>& sa) {
4     int n = s.size(), h = 0;
5     vector<int> r(n), l(n - 1);
6     for (int i = 0; i < n; i++)

```

lcp_array.cpp (cont.)

```

7   r[sa[i]] = i;
8   for (int i = 0; i < n; i++)
9       if (r[i]) {
10          int j = sa[r[i] - 1];
11          while (max(i, j) + h < n &&
12                s[i + h] == s[j + h])
13              h++;
14          l[r[i] - 1] = h;
15          if (h)
16              h--;
17      }
18  return l;
19 }

```

z_algorithm.cpp

```

1  template <class T>
2  vector<int> z_algorithm(const T& s) {
3      int n = s.size();
4      vector<int> z(n);
5      for (int i = 1, j = 0; i < n; i++) {
6          if (i < j + z[j])
7              z[i] = min(j + z[j] - i, z[i - j]);
8          while (i + z[i] < n &&
9                s[z[i]] == s[i + z[i]])
10             z[i]++;
11         if (i + z[i] > j + z[j])
12             j = i;
13     }
14     if (n)
15         z[0] = n;
16     return z;
17 }

```

rolling_hash.cpp

```

1  struct rolling_hash {
2      using ull = unsigned long long;
3      using u128 = __uint128_t;
4      static constexpr ull M = (1ULL << 61) - 1;
5      static ull mul(ull a, ull b) {
6          u128 x = (u128)a * b;
7          ull y = (x >> 61) + (x & M);
8          return y >= M ? y - M : y;
9      }
10     inline static ull B =
11         chrono::steady_clock::now()
12             .time_since_epoch()
13             .count() %
14             (M - 256) +
15             256;
16     vector<ull> h, p;
17     rolling_hash(const string& s)
18         : h(s.size() + 1),
19           p(s.size() + 1, 1) {
20         for (int i = 0; i < (int)s.size(); i++)
21             p[i + 1] = mul(p[i], B),
22             h[i + 1] =
23                 (mul(h[i], B) +
24                  (unsigned char)s[i] + 1) %
25                 M;
26     }
27     ull get(int l, int r) const {
28         ull x = h[r] + M - mul(h[l], p[r - l]);
29         return x >= M ? x - M : x;
30     }
31 };

```

scc.cpp

```

1  struct scc_graph {
2      int n;
3      vector<vector<int>> g, rg;
4      scc_graph(int n) : n(n), g(n), rg(n) {}
5      void add_edge(int a, int b) {
6          g[a].push_back(b);
7          rg[b].push_back(a);
8      }
9      vector<vector<int>> scc() {
10         vector<int> vis(n), ord;
11         for (int s = 0; s < n; s++)
12             if (!vis[s]) {
13                 stack<pair<int, int>> q;
14                 q.push({s, 0});
15                 vis[s] = 1;
16                 while (q.size()) {
17                     auto& [v, i] = q.top();
18                     if (i < (int)g[v].size()) {
19                         int u = g[v][i++];
20                         if (!vis[u])
21                             vis[u] = 1, q.push({u, 0});
22                     } else
23                         ord.push_back(v), q.pop();
24                 }
25             }
26         fill(vis.begin(), vis.end(), 0);
27         reverse(ord.begin(), ord.end());
28         vector<vector<int>> res;
29         for (int s : ord)
30             if (!vis[s]) {
31                 res.push_back({});
32                 stack<int> q;
33                 q.push(s);
34                 vis[s] = 1;
35                 while (q.size()) {
36                     int v = q.top();
37                     q.pop();
38                     res.back().push_back(v);
39                     for (int u : rg[v])
40                         if (!vis[u])
41                             vis[u] = 1, q.push(u);
42                 }
43             }
44         return res;
45     }
46 };

```

twosat.cpp

```

1  struct two_sat {
2      int n;
3      vector<vector<int>> g, rg;
4      vector<bool> a;
5      two_sat(int N)
6          : n(N), g(2 * N), rg(2 * N), a(N) {}
7
8      void add(int x, int y) {
9          g[x].push_back(y);
10         rg[y].push_back(x);
11     }
12     void add_clause(
13         int i, bool f, int j, bool h) {
14         add(2 * i + !f, 2 * j + h);
15         add(2 * j + !h, 2 * i + f);
16     }
17 }

```

twosat.cpp (cont.)

```

18 bool satisfiable() {
19     int N = 2 * n;
20     vector<int> v(N), o, c(N);
21     auto dfs = [&](auto&& self,
22                 int x) -> void {
23         v[x] = 1;
24         for (int y : g[x])
25             if (!v[y])
26                 self(self, y);
27         o.push_back(x);
28     };
29     for (int i = 0; i < N; i++)
30         if (!v[i])
31             dfs(dfs, i);
32
33     fill(v.begin(), v.end(), 0);
34     reverse(o.begin(), o.end());
35     int k = 0;
36     auto rdfs = [&](auto&& self,
37                    int x) -> void {
38         v[x] = 1;
39         c[x] = k;
40         for (int y : rg[x])
41             if (!v[y])
42                 self(self, y);
43     };
44     for (int x : o)
45         if (!v[x])
46             rdfs(rdfs, x), k++;
47
48     for (int i = 0; i < n; i++) {
49         if (c[2 * i] == c[2 * i + 1])
50             return false;
51         a[i] = c[2 * i] < c[2 * i + 1];
52     }
53     return true;
54 }
55
56 vector<bool> answer() { return a; }
57 };

```

rerooting.cpp

```

1 /*
2  - n 頂点の木 T が与えられている
3
4  - RerootingDP<E,V,merge,e,put_edge,
5    put_vertex> g(n) : コンストラクタ 0(n)
6    E : 親の dp の計算をする際に用いる値の
7    型
8    V : 頂点の dp の値の型
9    E put_edge (V v, int i) :
10     頂点 v を根とする部分木の dp の値に辺 i を追加し
11     た時に得られる値
12     E merge(E e1, E e2) : 2 つの子につい
13     て求めた put_edge の値を結合する
14     merge は可換モノイドによる結合を
15     要求される
16     E e() : merge の単位元を返す関数
17     V put_vertex(E e, int u) :
18     全ての子について merge した値から、頂点 u の dp
19     [u] を計算する
20     - void add_edge(int u, int v, int idx,
21     int xdi) : 頂点 u->v の辺番号 idx の辺と
22     頂点 v->u の辺番号 xdi の辺を追加する (
23     ちょうど n-1 回呼ぶ、 ならし 0(1))
24     - V bulid(int v) : 頂点 v を根とした時の、 d
25     p[v] の値を返す (1 回までしか呼べない 0(n))

```

rerooting.cpp (cont.)

```

15     - vector<V> reroot() : 長さ n の配列 (
16     i 番目には、頂点 i を根とした時の、 dp[
17     i] の値) を返す
18     (先に build をちょうど 1 回呼んでいる必
19     要あり 0(n))
20 */
21
22 template <class E, class V,
23           E (*merge)(E, E), E (*e)(),
24           E (*put_edge)(V, int),
25           V (*put_vertex)(E, int)>
26 struct RerootingDP {
27     struct edge {
28         int to, idx, xdi;
29     };
30
31     RerootingDP(int n_ = 0)
32         : n(n_), inner_edge_id(0) {
33         es.resize(2 * n - 2);
34         start.resize(2 * n - 2);
35         if (n == 1)
36             es_build();
37     }
38
39     void add_edge(
40         int u, int v, int idx, int xdi) {
41         start[inner_edge_id] = u;
42         es[inner_edge_id] = {v, idx, xdi};
43         inner_edge_id++;
44         start[inner_edge_id] = v;
45         es[inner_edge_id] = {u, xdi, idx};
46         inner_edge_id++;
47         if (inner_edge_id == 2 * n - 2) {
48             es_build();
49         }
50     }
51
52     vector<V> build(int root_ = 0) {
53         root = root_;
54         vector<V> subdp(n);
55         subdp[0] = put_vertex(e(), 0);
56         outs.resize(n);
57         vector<int> geta(n + 1, 0);
58         for (int i = 0; i < n; i++)
59             geta[i + 1] =
60                 start[i + 1] - start[i] - 1;
61         geta[root + 1]++;
62         for (int i = 0; i < n; i++)
63             geta[i + 1] += geta[i];
64         auto dfs = [&](auto& sfs, int v,
65                       int f) -> void {
66             E val = e();
67             for (int i = start[v];
68                  i < start[v + 1]; i++) {
69                 if (es[i].to == f) {
70                     swap(es[start[v + 1] - 1], es[i]);
71                 }
72                 if (es[i].to == f)
73                     continue;
74                 sfs(sfs, es[i].to, v);
75                 E nval = put_edge(
76                     subdp[es[i].to], es[i].idx);
77                 outs[geta[v]++] = nval;
78                 val = merge(val, nval);
79             }
80             subdp[v] = put_vertex(val, v);
81         };
82         dfs(dfs, root, -1);
83         return subdp;
84     }

```

rerooting.cpp (cont.)

```

79 vector<V> reroot() {
80     vector<E> reverse_edge(n);
81     reverse_edge[root] = e();
82     vector<V> answers(n);
83     auto dfs = [&](
84         auto sfs, int v) -> void {
85         int le = outs_start(v);
86         int ri = outs_start(v + 1);
87         int siz = ri - le;
88         vector<E> rui(siz + 1);
89         rui[siz] = e();
90         for (int i = siz - 1; i >= 0; i--) {
91             rui[i] =
92                 merge(outs[le + i], rui[i + 1]);
93         }
94         answers[v] = put_vertex(
95             merge(rui[0], reverse_edge[v]), v);
96         E lui = e();
97         for (int i = 0; i < siz; i++) {
98             V rdp = put_vertex(
99                 merge(merge(lui, rui[i + 1]),
100                     reverse_edge[v]),
101                 v);
102             reverse_edge[es[start[v] + i].to] =
103                 put_edge(
104                     rdp, es[start[v] + i].xdi);
105             lui = merge(lui, outs[le + i]);
106             sfs(sfs, es[start[v] + i].to);
107         }
108     };
109     dfs(dfs, root);
110     return answers;
111 }
112
113 private:
114 int n, root, inner_edge_id;
115 vector<E> outs;
116 vector<edge> es;
117 vector<int> start;
118 int outs_start(int v) {
119     int res = start[v] - v;
120     if (root < v)
121         res++;
122     return res;
123 }
124 void es_build() {
125     vector<edge> nes(2 * n - 2);
126     vector<int> nstart(n + 2, 0);
127     for (int i = 0; i < 2 * n - 2; i++)
128         nstart[start[i] + 2]++;
129     for (int i = 0; i < n; i++)
130         nstart[i + 1] += nstart[i];
131     for (int i = 0; i < 2 * n - 2; i++)
132         nes[nstart[start[i] + 1]++] = es[i];
133     swap(es, nes);
134     swap(start, nstart);
135 }
136 };

```

maxflow.cpp

```

1 template <class Cap>
2 struct mf_graph {
3     struct E {
4         int to, rev;
5         Cap cap;
6     };

```

maxflow.cpp (cont.)

```

7     struct edge {
8         int from, to;
9         Cap cap, flow;
10    };
11    int N;
12    vector<vector<E>> g;
13    vector<pair<int, int>> pos;
14
15    mf_graph(int n = 0) : N(n), g(n) {}
16
17    int add_edge(int s, int t, Cap c) {
18        int a = g[s].size(),
19            b = g[t].size() + (s == t);
20        pos.push_back({s, a});
21        g[s].push_back({t, b, c});
22        g[t].push_back({s, a, 0});
23        return (int)pos.size() - 1;
24    }
25
26    edge get_edge(int i) {
27        auto [v, j] = pos[i];
28        auto &e = g[v][j], &r = g[e.to][e.rev];
29        return {v, e.to, e.cap + r.cap, r.cap};
30    }
31
32    vector<edge> edges() {
33        vector<edge> r;
34        for (int i = 0; i < (int)pos.size();
35            i++)
36            r.push_back(get_edge(i));
37        return r;
38    }
39
40    void change_edge(int i, Cap c, Cap f) {
41        auto [v, j] = pos[i];
42        auto &e = g[v][j], &r = g[e.to][e.rev];
43        e.cap = c - f;
44        r.cap = f;
45    }
46
47    Cap flow(int s, int t) {
48        return flow(
49            s, t, numeric_limits<Cap>::max());
50    }
51
52    Cap flow(int s, int t, Cap lim) {
53        Cap ans = 0;
54        vector<int> lv(N), it(N);
55        while (ans < lim) {
56            fill(lv.begin(), lv.end(), -1);
57            queue<int> q;
58            q.push(s);
59            lv[s] = 0;
60            while (q.size()) {
61                int v = q.front();
62                q.pop();
63                for (auto& e : g[v])
64                    if (e.cap && lv[e.to] < 0)
65                        lv[e.to] = lv[v] + 1,
66                        q.push(e.to);
67            }
68            if (lv[t] < 0)
69                break;
70            fill(it.begin(), it.end(), 0);
71
72            auto dfs = [&](auto&& self, int v,

```

maxflow.cpp (cont.)

```

73         Cap f) -> Cap {
74     if (v == t)
75         return f;
76     for (int& i = it[v];
77          i < (int)g[v].size(); i++) {
78         E& e = g[v][i];
79         if (e.cap &&
80             lv[e.to] == lv[v] + 1)
81             if (Cap x = self(self, e.to,
82                             min(f, e.cap))) {
83                 e.cap -= x;
84                 g[e.to][e.rev].cap += x;
85                 return x;
86             }
87     }
88     return 0;
89 };
90
91 while (ans < lim) {
92     Cap x = dfs(dfs, s, lim - ans);
93     if (!x)
94         break;
95     ans += x;
96 }
97 }
98 return ans;
99 }
100
101 vector<bool> min_cut(int s) {
102     vector<bool> z(N);
103     queue<int> q;
104     q.push(s);
105     z[s] = 1;
106     while (q.size()) {
107         int v = q.front();
108         q.pop();
109         for (auto& e : g[v])
110             if (e.cap && !z[e.to])
111                 z[e.to] = 1, q.push(e.to);
112     }
113     return z;
114 }
115 };

```

mincostflow.cpp

```

1 template <class Cap, class Cost>
2 struct mcf_graph {
3     struct E {
4         int to, rev;
5         Cap cap;
6         Cost cost;
7     };
8     struct edge {
9         int from, to;
10        Cap cap, flow;
11        Cost cost;
12    };
13    int N;
14    vector<vector<E>> g;
15    vector<pair<int, int>> pos;
16
17    mcf_graph(int n = 0) : N(n), g(n) {}
18
19    int add_edge(
20        int s, int t, Cap c, Cost w) {
21        int a = g[s].size(),
22        b = g[t].size() + (s == t);

```

mincostflow.cpp (cont.)

```

23        pos.push_back({s, a});
24        g[s].push_back({t, b, c, w});
25        g[t].push_back({s, a, 0, -w});
26        return (int)pos.size() - 1;
27    }
28
29    edge get_edge(int i) {
30        auto [v, j] = pos[i];
31        auto &e = g[v][j], &r = g[e.to][e.rev];
32        return {v, e.to, e.cap + r.cap, r.cap,
33                e.cost};
34    }
35
36    vector<edge> edges() {
37        vector<edge> r;
38        for (int i = 0; i < (int)pos.size();
39             i++)
40            r.push_back(get_edge(i));
41        return r;
42    }
43
44    pair<Cap, Cost> flow(int s, int t) {
45        return flow(
46            s, t, numeric_limits<Cap>::max());
47    }
48
49    pair<Cap, Cost> flow(
50        int s, int t, Cap lim) {
51        return slope(s, t, lim).back();
52    }
53
54    vector<pair<Cap, Cost>> slope(
55        int s, int t) {
56        return slope(
57            s, t, numeric_limits<Cap>::max());
58    }
59
60    vector<pair<Cap, Cost>> slope(
61        int s, int t, Cap lim) {
62        const Cost INF =
63            numeric_limits<Cost>::max();
64        vector<Cost> h(N), d(N);
65        vector<int> pv(N), pe(N);
66
67        Cap f = 0;
68        Cost c = 0, last = -1;
69        vector<pair<Cap, Cost>> res{{0, 0}};
70
71        while (f < lim) {
72            fill(d.begin(), d.end(), INF);
73            d[s] = 0;
74
75            priority_queue<pair<Cost, int>,
76                        vector<pair<Cost, int>>,
77                        greater<pair<Cost, int>>>
78                q;
79            q.push({0, s});
80
81            while (q.size()) {
82                auto [dv, v] = q.top();
83                q.pop();
84                if (dv != d[v])
85                    continue;
86
87                for (int i = 0;

```

mincostflow.cpp (cont.)

```

88     i < (int)g[v].size(); i++) {
89         E& e = g[v][i];
90         if (!e.cap)
91             continue;
92
93         __int128 nd = (__int128)dv +
94             e.cost + h[v] - h[e.to];
95         if (nd < d[e.to]) {
96             d[e.to] = (Cost)nd;
97             pv[e.to] = v;
98             pe[e.to] = i;
99             q.push({(Cost)nd, e.to});
100         }
101     }
102 }
103
104 if (d[t] == INF)
105     break;
106
107 for (int v = 0; v < N; v++)
108     if (d[v] < INF)
109         h[v] += d[v];
110
111 Cap x = lim - f;
112 for (int v = t; v != s; v = pv[v])
113     x = min(x, g[pv[v]][pe[v]].cap);
114
115 for (int v = t; v != s; v = pv[v]) {
116     E& e = g[pv[v]][pe[v]];
117     e.cap -= x;
118     g[v][e.rev].cap += x;
119 }
120
121 Cost w = h[t] - h[s];
122 f += x;
123 c += x * w;
124
125 if (last == w)
126     res.pop_back();
127 res.push_back({f, c});
128 last = w;
129 }
130
131 return res;
132 }
133 };

```

math.cpp

```

1 using ll = long long;
2 using ull = unsigned long long;
3
4 ll smod(ll x, ll m) {
5     return (x % m) < 0 ? x + m : x;
6 }
7
8 pair<ll, ll> igcd(ll a, ll b) {
9     a = smod(a, b);
10    if (!a)
11        return {b, 0};
12    ll s = b, t = a, x = 0, y = 1;
13    while (t) {
14        ll q = s / t;
15        s -= t * q;
16        x -= y * q;
17        swap(s, t);
18        swap(x, y);
19    }

```

math.cpp (cont.)

```

20 if (x < 0)
21     x += b / s;
22 return {s, x};
23 }
24
25 ll pow_mod(ll x, ll n, int m) {
26     ll r = 1 % m;
27     x = smod(x, m);
28     for (; n; n >= 1, x = x * x % m)
29         if (n & 1)
30             r = r * x % m;
31     return r;
32 }
33
34 ll inv_mod(ll x, ll m) {
35     return igcd(x, m).second;
36 }
37
38 pair<ll, ll> crt(const vector<ll>& r,
39     const vector<ll>& m) {
40     ll R = 0, M = 1;
41     for (int i = 0; i < (int)r.size(); i++) {
42         ll x = smod(r[i], m[i]);
43         auto [g, im] = igcd(M, m[i]);
44         ll u = m[i] / g;
45         if ((x - R) % g)
46             return {0, 0};
47         ll t = (__int128)smod((x - R) / g, u) *
48             im % u;
49         R += M * t;
50         M *= u;
51         R = smod(R, M);
52     }
53     return {R, M};
54 }
55
56 ll floor_sum(ll n, ll m, ll a, ll b) {
57     ull z = 0;
58     if (a < 0) {
59         ull x = smod(a, m);
60         z -= 1ULL * n * (n - 1) / 2 *
61             ((x - a) / m);
62         a = x;
63     }
64     if (b < 0) {
65         ull x = smod(b, m);
66         z -= 1ULL * n * ((x - b) / m);
67         b = x;
68     }
69     for (;;) {
70         if (a >= m)
71             z += 1ULL * n * (n - 1) / 2 * (a / m),
72             a %= m;
73         if (b >= m)
74             z += 1ULL * n * (b / m), b %= m;
75         ull y = (ull)a * n + b;
76         if (y < (ull)m)
77             return (ll)z;
78         n = y / m;
79         b = y % m;
80         swap(a, m);
81     }
82 }

```

modint_convolution.cpp

modint_convolution.cpp (cont.)

```

1 struct mint {
2     static const int M = 998244353;
3     int x;
4     mint(long long v = 0)
5         : x((v % M + M) % M) {}
6     int val() const { return x; }
7     static int mod() { return M; }
8     mint& operator+=(mint a) {
9         if ((x += a.x) >= M)
10             x -= M;
11         return *this;
12     }
13     mint& operator-=(mint a) {
14         if ((x -= a.x) < 0)
15             x += M;
16         return *this;
17     }
18     mint& operator*=(mint a) {
19         x = (long long)x * a.x % M;
20         return *this;
21     }
22     mint pow(long long n) const {
23         mint a = *this, r = 1;
24         for (; n; n >>= 1, a *= a)
25             if (n & 1)
26                 r *= a;
27         return r;
28     }
29     mint inv() const { return pow(M - 2); }
30     mint& operator/=(mint a) {
31         return *this *= a.inv();
32     }
33     friend mint operator+(mint a, mint b) {
34         return a += b;
35     }
36     friend mint operator-(mint a, mint b) {
37         return a -= b;
38     }
39     friend mint operator*(mint a, mint b) {
40         return a *= b;
41     }
42     friend mint operator/(mint a, mint b) {
43         return a /= b;
44     }
45 };
46 using modint998244353 = mint;
47 void ntt(vector<mint>& a, bool inv = 0) {
48     int n = a.size();
49     for (int i = 1, j = 0; i < n; i++) {
50         int b = n >> 1;
51         for (; j & b; b >>= 1)
52             j ^= b;
53         j ^= b;
54         if (i < j)
55             swap(a[i], a[j]);
56     }
57     for (int l = 2; l <= n; l <= 1) {
58         mint w = mint(3).pow((mint::M - 1) / l);
59         if (inv)
60             w = w.inv();
61         for (int i = 0; i < n; i += l) {
62             mint z = 1;
63             for (int j = 0; j < l / 2; j++) {
64                 mint x = a[i + j],
65                     y = a[i + j + l / 2] * z;
66                 a[i + j] = x + y;
67                 a[i + j + l / 2] = x - y;

```

modint_convolution.cpp (cont.)

```

68         z *= w;
69     }
70 }
71 }
72 if (inv) {
73     mint z = mint(n).inv();
74     for (auto& x : a)
75         x *= z;
76 }
77 }
78 vector<mint> convolution(
79     vector<mint> a, vector<mint> b) {
80     if (a.empty() || b.empty())
81         return {};
82     int n = a.size() + b.size() - 1, z = 1;
83     while (z < n)
84         z <= 1;
85     a.resize(z);
86     b.resize(z);
87     ntt(a);
88     ntt(b);
89     for (int i = 0; i < z; i++)
90         a[i] *= b[i];
91     ntt(a, 1);
92     a.resize(n);
93     return a;
94 }

```

geometry.cpp

```

1 using ld = long double;
2 using Point = complex<ld>;
3 using pt = Point;
4 const ld EPS = 1e-10L, PI = acos(-1.L);
5
6 inline bool equal(const ld& a,
7     const ld& b) { //a,b が誤差を許して等しいか
8     return fabs(a - b) < EPS;
9 }
10 inline int sgn(
11     const ld& x) { //x の符号を-1,0,1 で返す
12     return (x > EPS) - (x < -EPS);
13 }
14 Point unitVector(const Point&
15     a) { //a と同方向の単位ベクトル
16     assert(abs(a) > EPS);
17     return a / abs(a);
18 }
19 Point normalVector(const Point&
20     a) { //a を反時計回りに 90 度回転
21     return a * Point(0, 1);
22 }
23 ld dot(
24     const Point& a, const Point& b) { //内積
25     return real(conj(a) * b);
26 }
27 ld cross(
28     const Point& a, const Point& b) { //外積
29     return imag(conj(a) * b);
30 }
31 Point rotate(const Point& p,
32     ld t) { //p を反時計回りに t(rad)回転
33     return p * polar<ld>(1, t);
34 }
35 ld radianToDegree(ld x) { //rad を度に変換
36     return x * 180 / PI;
37 }
38 ld degreeToRadian(ld x) { //度を rad に変換

```

geometry.cpp (cont.)

```

39  return x * PI / 180;
40 }
41
42 struct Line {
43     Point a, b;
44     Line() = default;
45     Line(Point x, Point y)
46         : a(x), b(y) {}
47 } //2点x,yを通る直線(線分)
48 Line(
49     ld A, ld B, ld C) { //Ax+By=Cを表す直線
50     assert(!equal(A, 0) || !equal(B, 0));
51     if (!equal(B, 0))
52         a = {0, C / B}, b = {1, (C - A) / B};
53     else
54         a = {C / A, 0}, b = {C / A, 1};
55 };
56
57 struct Circle {
58     Point p;
59     ld r;
60     Circle() = default;
61     Circle(Point x, ld y)
62         : p(x), r(y) {} //中心x,半径yの円
63 };
64
65 bool onLine(const Line& l,
66     const Point& p) { //pが直線l上か
67     return fabsl(cross(l.b - l.a, p - l.a)) <=
68         EPS;
69 }
70
71 bool onSegment(const Line& l,
72     const Point& p) { //pが線分l上か
73     return ccw(l.a, l.b, p) == 0;
74 }
75
76 Circle circumCircle(Point a, Point b,
77     Point c) { //三角形abcの外接円
78     assert(fabsl(cross(b - a, c - a)) > EPS);
79     Point x = (a + b) / 2, y = (a + c) / 2;
80     Line l = {x, x + normalVector(b - a)};
81     Line m = {y, y + normalVector(c - a)};
82     Point p = crossPoint(l, m);
83     return {p, abs(p - a)};
84 }
85
86 Point projection(const Line& l,
87     const Point&
88     p) { //lへpから下ろした垂線の足
89     Point d = l.b - l.a;
90     assert(abs(d) > EPS);
91     return l.a +
92         d * dot(p - l.a, d) / norm(d);
93 }
94
95 Point reflection(const Line& l,
96     const Point& p) { //lに関するpの対称点
97     return 2 * l * projection(l, p) - p;
98 }
99
100 int ccw(const Point& a, Point b,
101     Point c) { //a,b,cの位置関係を判定
102     b -= a;
103     c -= a;
104     ld x = cross(b, c);
105     if (abs(x) > EPS)

```

geometry.cpp (cont.)

```

106     return x > 0 ? 1 : -1; //反時計/時計
107     if (dot(b, c) < -EPS)
108         return 2; //c-a-b
109     if (norm(b) + EPS < norm(c))
110         return -2; //a-b-c
111     return 0; //cが線分ab上
112 }
113
114 bool isOrthogonal(const Line& a,
115     const Line& b) { //2直線が直交するか
116     return equal(
117         dot(a.b - a.a, b.b - b.a), 0);
118 }
119 bool isParallel(const Line& a,
120     const Line& b) { //2直線が平行か
121     return equal(
122         cross(a.b - a.a, b.b - b.a), 0);
123 }
124 bool isIntersect(const Line& a,
125     const Line& b) { //2線分が交差するか
126     return ccw(a.a, a.b, b.a) *
127         ccw(a.a, a.b, b.b) <=
128         0 &&
129         ccw(b.a, b.b, a.a) *
130         ccw(b.a, b.b, a.b) <=
131         0;
132 }
133
134 Point crossPoint(const Line& a,
135     const Line& b) { //2直線の交点
136     Point d = a.b - a.a, e = b.b - b.a;
137     ld z = cross(d, e),
138         w = cross(d, b.a - a.a);
139     if (equal(z, 0)) {
140         assert(equal(w, 0));
141         return b.a;
142     }
143     return a.a + d * cross(b.a - a.a, e) / z;
144 }
145
146 ld distanceBetweenLineAndPoint(
147     const Line& l,
148     const Point& p) { //直線lと点pの距離
149     Point d = l.b - l.a;
150     assert(abs(d) > EPS);
151     return fabsl(cross(d, p - l.a)) / abs(d);
152 }
153 ld distanceBetweenSegmentAndPoint(
154     const Line& l,
155     const Point& p) { //線分lと点pの距離
156     Point d = l.b - l.a;
157     assert(abs(d) > EPS);
158     if (dot(d, p - l.a) < EPS)
159         return abs(p - l.a);
160     if (dot(-d, p - l.b) < EPS)
161         return abs(p - l.b);
162     return fabsl(cross(d, p - l.a)) / abs(d);
163 }
164 ld distanceBetweenSegments(const Line& a,
165     const Line& b) { //2線分間の距離
166     if (isIntersect(a, b))
167         return 0;

```

geometry.cpp (cont.)

```

168 return min(
169     {distanceBetweenSegmentAndPoint(a, b.a),
170     distanceBetweenSegmentAndPoint(
171         a, b.b),
172     distanceBetweenSegmentAndPoint(
173         b, a.a),
174     distanceBetweenSegmentAndPoint(
175         b, a.b)});
176 }
177
178 ld SignedPolygonArea(const vector<Point>&
179     p) { //多角形の符号付き面積
180     ld s = 0;
181     for (int i = 0, n = p.size(); i < n; i++)
182         s += cross(p[i], p[(i + 1) % n]);
183     return s / 2;
184 }
185 ld PolygonArea(
186     const vector<Point>& p) { //多角形の面積
187     return fabs1(SignedPolygonArea(p));
188 }
189
190 bool isConvex(
191     const vector<Point>& p) { //多角形が凸か
192     if (p.size() < 3)
193         return false;
194     int s = 0;
195     for (int i = 0, n = p.size(); i < n;
196         i++) {
197         int t = sgn(cross(p[(i + 1) % n] - p[i],
198             p[(i + 2) % n] - p[(i + 1) % n]));
199         if (t && s && t != s)
200             return false;
201         if (t)
202             s = t;
203     }
204     return true;
205 }
206
207 int isContained(const vector<Point>& g,
208     const Point&
209     p) { //点 p が多角形 g の内/辺上/外か(2/1/0)
210     bool in = 0;
211     for (int i = 0, n = g.size(); i < n;
212         i++) {
213         Point a = g[i] - p,
214             b = g[(i + 1) % n] - p;
215         if (imag(a) > imag(b))
216             swap(a, b);
217         if (imag(a) <= EPS && EPS < imag(b) &&
218             cross(a, b) < -EPS)
219             in ^= 1;
220         if (equal(cross(a, b), 0) &&
221             dot(a, b) <= EPS)
222             return 1;
223     }
224     return in ? 2 : 0;
225 }
226
227 vector<Point> ConvexHull(vector<Point> p,
228     bool col =
229     false) { //凸包を反時計回りに返す(
230     ↪ col=trueなら辺上も含む)

```

geometry.cpp (cont.)

```

230 sort(p.begin(), p.end(),
231     [](Point a, Point b) {
232         return real(a) != real(b)
233             ? real(a) < real(b)
234             : imag(a) < imag(b);
235     });
236 p.erase(unique(p.begin(), p.end(),
237     [](Point a, Point b) {
238         return abs(a - b) < EPS;
239     }),
240     p.end());
241
242 int n = p.size(), k = 0;
243 if (n <= 1)
244     return p;
245 vector<Point> h(2 * n);
246 if (col) {
247     bool all = 1;
248     for (int i = 2; i < n; i++)
249         if (fabs1(cross(p[1] - p[0],
250             p[i] - p[0])) > EPS)
251             all = 0;
252     if (all)
253         return p;
254 }
255
256 auto bad =
257     [&](Point a, Point b,
258         Point c) { //凸包に不要な中間点か
259         ld x = cross(b - a, c - b);
260         return col ? x < -EPS : x < EPS;
261     };
262
263 for (auto x : p) {
264     while (
265         k >= 2 && bad(h[k - 2], h[k - 1], x))
266         --k;
267     h[k++] = x;
268 }
269 for (int i = n - 2, t = k + 1; i >= 0;
270     i--) {
271     while (k >= t &&
272         bad(h[k - 2], h[k - 1], p[i]))
273         --k;
274     h[k++] = p[i];
275 }
276 h.resize(k - 1);
277 return h;
278 }
279
280 int isIntersect(const Circle& a,
281     const Circle&
282     b) { //2円の位置関係(返り値=共通接線数,
283     ↪ -1は同一円)
284     ld d = abs(a.p - b.p), s = a.r + b.r,
285         t = fabs1(a.r - b.r);
286     if (equal(d, 0) && equal(a.r, b.r))
287         return -1;
288     if (d > s + EPS)
289         return 4; //離れている
290     if (equal(d, s))
291         return 3; //外接
292     if (d < t - EPS)
293         return 0; //内包
294     if (equal(d, t))
295         return 1; //内接

```

geometry.cpp (cont.)

```

295 return 2; //2 点で交わる
296 }
297
298 Circle inCircle(const Point& a,
299 const Point& b,
300 const Point& c) { //三角形 abc の内接円
301 ld A = abs(b - c), B = abs(a - c),
302 C = abs(a - b);
303 assert(A + B + C > EPS &&
304 fabsl(cross(b - a, c - a)) > EPS);
305 Point p =
306 (a * A + b * B + c * C) / (A + B + C);
307 return {p,
308 distanceBetweenLineAndPoint({a, b}, p)};
309 }
310
311 vector<Point> crossPoint(const Circle& c,
312 const Line& l) { //円 c と直線 l の交点
313 ld d =
314 distanceBetweenLineAndPoint(l, c.p);
315 if (d > c.r + EPS)
316 return {};
317 Point h = projection(l, c.p);
318 if (equal(d, c.r))
319 return {h};
320 Point u = unitVector(l.b - l.a) *
321 sqrt(max<ld>(0, c.r * c.r - d * d));
322 return {h - u, h + u};
323 }
324
325 vector<Point> crossPoint(const Circle& a,
326 const Circle& b) { //2 円の交点
327 int m = isIntersect(a, b);
328 if (m == 4 || m == 0 || m == -1)
329 return {};
330
331 ld d = abs(b.p - a.p);
332 ld x = (a.r * a.r + d * d - b.r * b.r) /
333 (2 * d);
334 ld y =
335 sqrt(max<ld>(0, a.r * a.r - x * x));
336
337 Point u = (b.p - a.p) / d,
338 h = a.p + u * x;
339 if (m == 1 || m == 3)
340 return {h};
341
342 Point v = u * Point(0, y);
343 return {h + v, h - v};
344 }
345
346 vector<Point> tangentToCircle(
347 const Point& p,
348 const Circle& c) { //点 p から円 c への接点
349 ld d = norm(c.p - p);
350 if (d < c.r * c.r - EPS)
351 return {};
352 return crossPoint(c,
353 Circle(
354 p, sqrt(max<ld>(0, d - c.r * c.r))));
355 }
356
357 vector<Line> tangent(const Circle& a,
358 const Circle& b) { //2 円の共通接線
359 vector<Line> r;
360 ld g = abs(a.p - b.p);
361 if (equal(g, 0))

```

geometry.cpp (cont.)

```

362 return r;
363
364 Point u = unitVector(b.p - a.p),
365 v = normalVector(u);
366
367 for (int s : {-1, 1}) {
368 ld h = (a.r + b.r * s) / g,
369 q = 1 - h * h;
370 if (q < -EPS)
371 continue;
372
373 if (fabsl(q) < EPS) {
374 Point w = h > 0 ? u : -u;
375 r.push_back(
376 {a.p + w * a.r, a.p + w * a.r + v});
377 } else {
378 Point U = u * h,
379 V = v * sqrt(max<ld>(0, q));
380 for (int t : {1, -1}) {
381 Point w = U + V * (ld)t;
382 r.push_back({a.p + w * a.r,
383 b.p - w * (b.r * s)});
384 }
385 }
386 }
387 return r;
388 }

```

cartesiantree.cpp

```

1 template <class T>
2 struct CartesianTree {
3 int root = 0; //最小値のうち index 最小が根
4 vector<int> par, left, right;
5
6 CartesianTree() {}
7 CartesianTree(const vector<T>&
8 a) // a の Cartesian Tree を O(n) で構築
9 : par(a.size(), -1),
10 left(a.size(), -1),
11 right(a.size(), -1) {}
12 vector<int> st;
13 for (int i = 0; i < (int)a.size();
14 i++) {
15 int p = -1;
16 while (
17 !st.empty() && a[st.back()] > a[i])
18 p = st.back(), st.pop_back();
19 if (!st.empty())
20 par[i] = st.back(),
21 right[st.back()] = i;
22 if (p >= 0)
23 par[p] = i, left[i] = p;
24 st.push_back(i);
25 }
26 if (!st.empty())
27 root = st[0];
28 }
29 };

```

convex_hull_trick.cpp

```

1 template <class T, const T id>
2 struct convex_hull_trick {
3 struct L {
4 T a = 0, b = id;
5 T f(T x) const {
6 return a * x + b;
7 } // 直線 ax+b の x での値
8 };

```

convex_hull_trick.cpp (cont.)

```

9
10 int n;
11 vector<T> x;
12 vector<L> t;
13 vector<char> u;
14
15 convex_hull_trick(const vector<T>&
16     X) // 使用する x 座標を渡して構築
17     : n(X.size()), x(X), t(4 * n),
18     u(4 * n) {}
19
20 void insert(
21     T a, T b) { // 直線 ax+b を追加
22     add(1, 0, n - 1, {a, b});
23 }
24
25 T get(T X)
26 { const { // x=X での追加済み直線の最大値
27     int p =
28         lower_bound(x.begin(), x.end(), X) -
29         x.begin();
30     assert(p < n && x[p] == X);
31     return qry(1, 0, n - 1, p);
32 }
33
34 private:
35 void add(int k, int l, int r,
36     L z) { // Li Chao Tree に直線 z を追加
37     if (!u[k]) {
38         t[k] = z;
39         u[k] = 1;
40         return;
41     }
42     int m = (l + r) / 2;
43     bool a = z.f(x[l]) > t[k].f(x[l]);
44     bool b = z.f(x[m]) > t[k].f(x[m]);
45     if (b)
46         swap(z, t[k]);
47     if (l == r)
48         return;
49     if (a != b)
50         add(2 * k, l, m, z);
51     else
52         add(2 * k + 1, m + 1, r, z);
53 }
54
55 T qry(int k, int l, int r,
56     int p) const { // x[p] での最大値を求める
57     if (!u[k])
58         return id;
59     T y = t[k].f(x[p]);
60     if (l == r)
61         return y;
62     int m = (l + r) / 2;
63     return max(y,
64         p <= m ? qry(2 * k, l, m, p)
65         : qry(2 * k + 1, m + 1, r, p));
66 }
67 };

```

dynamic_segtree.cpp

```

1 template <class S, auto op, auto e>
2 class dynamic_segtree {
3 private:
4     struct node;
5     using node_ptr = unique_ptr<node>;

```

dynamic_segtree.cpp (cont.)

```

6     struct node {
7         long long idx;
8         S value, product;
9         node_ptr left, right;
10        node(long long idx, S value)
11            : idx(idx), value(value),
12            product(value), left(nullptr),
13            right(nullptr) {}
14
15        void update() {
16            product =
17                op(op(left ? left->product : e(),
18                    value),
19                    right ? right->product : e());
20        }
21    };
22    const long long n;
23    node_ptr root;
24
25 public:
26    dynamic_segtree() : dynamic_segtree(0) {}
27    dynamic_segtree(long long n)
28        : n(n), root(nullptr) {}
29
30    void set(long long p, S x) {
31        assert(0 <= p && p < n);
32        set(root, 0, n, p, x);
33    }
34    void set(node_ptr& t, long long a,
35        long long b, long long p, S x) {
36        if (!t) {
37            t = make_unique<node>(p, x);
38            return;
39        }
40        if (t->idx == p) {
41            t->value = x;
42            t->update();
43            return;
44        }
45        long long c = (a + b) / 2;
46        if (p < c) {
47            if (t->idx < p)
48                swap(t->idx, p), swap(t->value, x);
49            set(t->left, a, c, p, x);
50        } else {
51            if (p < t->idx)
52                swap(p, t->idx), swap(x, t->value);
53            set(t->right, c, b, p, x);
54        }
55        t->update();
56    }
57
58    S get(long long p) const {
59        assert(0 <= p && p < n);
60        return get(root, 0, n, p);
61    }
62
63    S get(const node_ptr& t, long long a,
64        long long b, long long p) const {
65        if (!t)
66            return e();
67        if (t->idx == p)
68            return t->value;
69        long long c = (a + b) / 2;
70        if (p < c)
71            return get(t->left, a, c, p);
72        else

```

dynamic_segtree.cpp (cont.)

```

73     return get(t->right, c, b, p);
74 }
75
76 S prod(long long l, long long r) {
77     assert(0 <= l && l <= r && r <= n);
78     return prod(root, 0, n, l, r);
79 }
80
81 S prod(const node_ptr& t, long long a,
82     long long b, long long l,
83     long long r) const {
84     if (!t || b <= l || r <= a)
85         return e();
86     if (l <= a && b <= r)
87         return t->product;
88     long long c = (a + b) / 2;
89     S result = prod(t->left, a, c, l, r);
90     if (l <= t->idx && t->idx < r)
91         result = op(result, t->value);
92     return op(
93         result, prod(t->right, c, b, l, r));
94 }
95
96 S all_prod() const {
97     return root ? root->product : e();
98 }
99 };

```

eulertour.cpp

```

1 struct EulerTour {
2     int N;
3     vector<int> vs, depth, id, id_depth,
4         id_rev;
5     vector<vector<int>>> G;
6     EulerTour(int n, vector<vector<int>>>& g) {
7         N = n;
8         G = g;
9         vs.resize(2 * N - 1); //DFS での訪問順
10        depth.resize(
11            2 * N - 1); // 訪問順での根からの深さ
12        id.resize(
13            N); //id[i]:iが初めて現れる index
14        id_depth.resize(N); //頂点 i の深さ
15        id_rev.resize(N); //i が最後に現れる index
16        int k = 0;
17        dfs(0, -1, 0, k);
18    }
19    void dfs(
20        int now, int par, int d, int& k) {
21        id[now] = k;
22        id_rev[now] = k;
23        id_depth[now] = d;
24        vs[k] = now;
25        depth[k++] = d;
26        for (auto e : G[now]) {
27            if (e == par)
28                continue;
29            dfs(e, now, d + 1, k);
30            id_rev[now] = k;
31            vs[k] = now;
32            depth[k++] = d;
33        }
34    }
35 };
36 //int lca(int u,int v){ return vs[seg.prod(
37     min(id[u],id[v]),max(id[u],id[v])+1).idx];
38 }

```

hld.cpp

```

1 template <typename Graph>
2 struct HeavyLightDecomposition {
3     //Edge query:: seg.set(hld.get_edge(u[i],
4     v[i]),w[i]);
5     //auto f=[&](ll u,ll v){ans+=seg.prod(u,
6     v);return;};
7     //hld.path_query<function<void(ll,ll)>>(
8     uc,vc,false,f);
9 private:
10    void dfs_sz(int now, int p) {
11        // 1st dfs
12        par[now] = p;
13        depth[now] =
14            (p == -1 ? 0 : depth[p] + 1);
15        size[now] = 1;
16        if (G[now].size() && G[now][0] == p)
17            swap(G[now][0], G[now].back());
18        for (auto& e : G[now]) {
19            if (e == p)
20                continue;
21            dfs_sz(e, now);
22            size[now] += size[e];
23            if (size[e] > size[G[now][0]]) {
24                swap(G[now][0], e);
25            }
26        }
27    }
28    void dfs_hld(int now, int p) {
29        id[now] = id_k;
30        vertex[id_k++] = now;
31        for (auto& e : G[now]) {
32            if (e == p)
33                continue;
34            bool heavy = (e == G[now][0]);
35            head[e] = (heavy ? head[now] : e);
36            dfs_hld(e, now);
37        }
38    }
39    vector<pair<int, int>> ascend(
40        int u, int v) const {
41        vector<pair<int, int>> res;
42        while (head[u] != head[v]) {
43            res.push_back({id[u], id[head[u]]});
44            u = par[head[u]];
45        }
46        if (u != v)
47            res.push_back({id[u], id[v] + 1});
48        return res;
49    }
50    vector<pair<int, int>> descend(
51        int u, int v) const {
52        if (u == v)
53            return {};
54        if (head[u] == head[v])
55            return {{id[u] + 1, id[v]}};
56        auto res = descend(u, par[head[v]]);
57        res.push_back({id[head[v]], id[v]});
58        return res;
59    }
60 public:
61    Graph& G;
62    int root, id_k;
63    vector<int> size, par, depth, head, id,
64        vertex;

```

hld.cpp (cont.)

```

64 //head[v]:=vを通る heavy path のうち最も根側
   へ にある頂点
65 //id[v]:=v を行きがけ順で何番目に通るか
66 //vertex[i]:=行きがけ順で i 番目に通る頂点番
   へ 号
67 HeavyLightDecomposition(
68     Graph& _g, int _root = 0)
69     : G(_g), root(_root), id_k(0),
70       size(G.size(), 0),
71       par(G.size(), root),
72       depth(G.size(), 0),
73       head(G.size(), 0), id(G.size(), 0),
74       vertex(G.size(), 0) {
75     dfs_sz(root, -1);
76     dfs_hld(root, -1);
77 }
78
79 template <typename F>
80 void path_query(
81     int u, int v, bool vertex, const F& f) {
82     int l = lca(u, v);
83     for (auto [l, r] : ascend(u, l)) {
84         int s = l + 1, t = r;
85         s > t ? f(t, s) : f(s, t);
86     }
87     if (vertex)
88         f(id[l], id[l] + 1);
89     for (auto [l, r] : descend(l, v)) {
90         int s = l, t = r + 1;
91         s > t ? f(t, s) : f(s, t);
92     }
93 }
94
95 template <typename F>
96 void path_noncommutative_query(
97     int u, int v, bool vertex, const F& f) {
98     int l = lca(u, v);
99     for (auto [l, r] : ascend(u, l))
100         f(l + 1, r);
101     if (vertex)
102         f(id[l], id[l] + 1);
103     for (auto [l, r] : descend(l, v))
104         f(l, r + 1);
105 }
106
107 int level_ancestor(int v, int d) {
108     //v の祖先で深さ d にあるものは?
109     assert(depth[v] >= d);
110     while (depth[head[v]] > d) {
111         v = par[head[v]];
112     }
113     return vertex[id[v] - (depth[v] - d)];
114 }
115
116 int lca(int u, int v) {
117     while (head[u] != head[v]) {
118         if (id[u] > id[v])
119             swap(u, v);
120         v = par[head[v]];
121     }
122     return (id[u] < id[v] ? u : v);
123 }
124
125 int dist(int u, int v) {
126     return depth[u] + depth[v] -
127         2 * depth[lca(u, v)];
128 }

```

hld.cpp (cont.)

```

129 int get_edge(int u, int v) {
130     if (abs(depth[u] - depth[v]) != 1)
131         return -1;
132     if (depth[u] > depth[v])
133         return id[u];
134     return id[v];
135 }
136 };

```

linear_equation.cpp

```

1 // 行列 a をガウス消去する
2 // 返回值 : {rank, det}
3 // rank : pivot_end 列目までを係数部分とし
   へ  たときの rank
4 // det : 消去中の pivot の積×行交換の符号
5 // 正方形行列 a に対して全列を pivot 対象
   へ  にした場合は det(a)
6 // diagonalize=true なら簡約行階段形にする
7 template <class T>
8 pair<int, T> GaussElimination(
9     vector<vector<T>>& a, int pe = -1,
10     bool diag = false) {
11     if (a.empty())
12         return {0, T(1)};
13     int h = a.size(), w = a[0].size(), r = 0;
14     if (pe < 0)
15         pe = w;
16     T det = 1;
17     for (int j = 0; j < pe; j++) {
18         int p = r;
19         while (p < h && a[p][j] == T(0))
20             p++;
21         if (p == h) {
22             det = 0;
23             continue;
24         }
25         if (p != r)
26             swap(a[p], a[r]), det = -det;
27         det *= a[r][j];
28         if (diag) {
29             T x = T(1) / a[r][j];
30             for (int k = j; k < w; k++)
31                 a[r][k] *= x;
32         }
33         for (int i = diag ? 0 : r + 1; i < h; i++)
34             if (i != r && a[i][j] != T(0)) {
35                 T x = a[i][j] / a[r][j];
36                 for (int k = j; k < w; k++)
37                     a[i][k] -= a[r][k] * x;
38             }
39         r++;
40     }
41     return {r, det};
42 }
43
44 // 線形方程式 Ax=b を解く
45 // 返回值 : 解なしなら {}
46 // 解ありなら {v, w1, ..., wk}
47 // ただし全ての解は
48 // x = v + c1*w1 + ... + ck*wk
49 // と表される
50 // v : 特殊解の 1 つ
51 // w1..wk : Ker(A) の基底 (自由変数に対応
   へ )
52 // k : W-rank(A) = 解空間の次元

```

linear_equation.cpp (cont.)

```

54 template <class T>
55 vector<vector<T>> LinearEquation(
56     vector<vector<T>> a, vector<T> b) {
57     int h = a.size(), w = a[0].size();
58     for (int i = 0; i < h; i++)
59         a[i].push_back(b[i]);
60     int r =
61         GaussElimination(a, w, true).first;
62     for (int i = r; i < h; i++)
63         if (a[i][w] != T(0))
64             return {};
65
66     vector<vector<T>> res(1, vector<T>(w));
67     vector<int> p(w, -1);
68     for (int i = 0, j = 0; i < r; i++) {
69         while (a[i][j] == T(0))
70             j++;
71         p[j] = i;
72         res[0][j] = a[i][w];
73     }
74     for (int j = 0; j < w; j++)
75         if (p[j] < 0) {
76             vector<T> x(w);
77             x[j] = 1;
78             for (int k = 0; k < j; k++)
79                 if (p[k] >= 0)
80                     x[k] = -a[p[k]][j];
81             res.push_back(x);
82         }
83     return res;
84 }

```

linear_equation_bitset.cpp

```

1 // F_2 上の線形方程式  $Ax=b$  を解く
2 // a[i][0..W-1] : A の i 行目, a[i][W] : b[i]
3 // 返回值 : 解なしなら {}
4 //          解ありなら {v, w1, ..., wk}
5 //          全解は  $x=v+c1*w1+...+ck*wk$  (ci
6 //          in F_2)
7 //          v : 特殊解
8 //          w1..wk : Ker(A)の基底 0(HW^2/64)
9 template <int MAX>
10 vector<bitset<MAX>> LinearEquation(
11     vector<bitset<MAX + 1>> a, int W) {
12     int H = a.size(), r = 0;
13     vector<int> p(W, -1);
14     for (int j = 0; j < W; j++) {
15         int q = r;
16         while (q < H && !a[q][j])
17             q++;
18         if (q == H)
19             continue;
20         swap(a[r], a[q]);
21         p[j] = r;
22         for (int i = 0; i < H; i++)
23             if (i != r && a[i][j])
24                 a[i] ^= a[r];
25         r++;
26     }
27     for (int i = r; i < H; i++)
28         if (a[i][W])
29             return {};
30     vector<bitset<MAX>> res(1);
31     for (int j = 0; j < W; j++)
32         if (p[j] >= 0)
33             res[0][j] = a[p[j]][W];

```

linear_equation_bitset.cpp (cont.)

```

34
35     for (int j = 0; j < W; j++)
36         if (p[j] < 0) {
37             bitset<MAX> x;
38             x[j] = 1;
39             for (int k = 0; k < W; k++)
40                 if (p[k] >= 0 && a[p[k]][j])
41                     x[k] = 1;
42             res.push_back(x);
43         }
44     return res;
45 }

```

manacher.cpp

```

1 template <class T>
2 vector<int> manacher(const T& s) {
3     int n = s.size();
4     vector<int> R(n);
5     int i = 0, j = 0;
6     while (i < s.size()) {
7         while (i - j >= 0 && i + j < s.size() &&
8             s[i - j] == s[i + j])
9             ++j;
10        R[i] = j;
11        int k = 1;
12        while (i - k >= 0 && k + R[i - k] < j)
13            R[i + k] = R[i - k], ++k;
14        i += k;
15        j -= k;
16    }
17    return R;
18 }

```

mo.cpp

```

1 struct Mo {
2     int n, l = 0, r = 0; // 現在の区間[l,r)
3     vector<pair<int, int>>
4         lr; // [l,r) addやeraseでは更新後のl,
5         rを参照できる
6     Mo(int N) : n(N) {} // i 番目の答えが入る
7     void add(
8         int l, int r) { // [l,r) のクエリを追加
9         lr.emplace_back(l, r);
10    }
11    template <class AL, class AR, class EL,
12        class ER, class O>
13    void build(const AL& al, const AR& ar,
14        const EL& el, const ER& er,
15        const O& out) {
16        int Q = lr.size();
17        if (!Q)
18            return;
19        int B = max(1, n / (int)sqrt(Q));
20        vector<int> o(Q);
21        iota(o.begin(), o.end(), 0);
22        sort(o.begin(), o.end(),
23            [&](int a, int b) {
24                int x = lr[a].first / B,
25                    y = lr[b].first / B;
26                return x != y
27                    ? x < y
28                    : (x & 1 ? lr[a].second >
29                        lr[b].second
30                        : lr[a].second <
31                        lr[b].second);
32            });
33        l = r = 0;

```

mo.cpp (cont.)

```

33     for (int i : o) {
34         auto [L, R] = lr[i];
35         while (l > L)
36             al(--l);
37         while (r < R)
38             ar(r++);
39         while (l < L)
40             el(l++);
41         while (r > R)
42             er(--r);
43         out(i);
44     }
45 }
46 template <class A, class E, class O>
47 void build(const A& add, const E& erase,
48           const O& out) { //add(i):
49     build(add, add, erase, erase, out);
50 }
51 };
52 // auto add_left = [&](int i){};
53 // auto erase = [&](int i){};
54 // auto out = [&](int q){}; 現在のクエリ(
55 //   q 番目)の答えを求める

```

rollback_mo.cpp

```

1 struct MoRollBack {
2     int w, l = 0, r = 0; // 現在の区間 [l,r)
3     vector<pair<int, int>> q;
4     vector<int> ord;
5     MoRollBack(int N, int Q)
6         : w(max(1, (int)sqrt(max(1, N)))),
7           ord(Q) {
8         iota(ord.begin(), ord.end(), 0);
9     }
10    void add(
11        int L, int R) { // クエリ [L,R)を追加
12        q.push_back({L, R});
13    }
14    template <class A, class REM, class RS,
15            class S, class B>
16    void run(const A& add, const REM& rem,
17            const RS& reset, const S& snapshot,
18            const B& rollback) { // 全クエリを処理
19        assert(q.size() == ord.size());
20        sort(ord.begin(), ord.end(),
21             [&](int i, int j) {
22                 int a = q[i].first / w,
23                     b = q[j].first / w;
24                 return a != b
25                     ? a < b
26                     : q[i].second < q[j].second;
27             });
28        l = r = 0;
29        reset();
30        for (int i : ord)
31            if (q[i].second - q[i].first < w) {
32                auto [L, R] = q[i];
33                l = r = L;
34                while (r < R) {
35                    int j = r++;
36                    add(
37                        j); // add 後の区間 [l,r)
38                }
39                rem(
40                    i); // [l,r) はクエリ区間そのもの

```

rollback_mo.cpp (cont.)

```

41        l = r = 0;
42        rollback(); // rollback 後は空区間 [
43        0,0)
44    }
45    int last = -1;
46    for (int i : ord)
47        if (q[i].second - q[i].first >= w) {
48            auto [L, R] = q[i];
49            int b = L / w, base = (b + 1) * w;
50            if (b != last) {
51                last = b;
52                l = r = base;
53                reset(); // 空区間 [base,base)
54            }
55            while (r < R) {
56                int j = r++;
57                add(j); // 右に追加後
58            }
59            snapshot();
60            while (l > L)
61                add(--l); // 左に追加後
62            rem(i); // [l,r)=[L,R)
63            l = base;
64            rollback(); // [base,r) まで戻す
65        }
66    };
67    /* 区間内の異なる値の個数
68    int N,Q;
69    cin>>N>>Q;
70    vector<int>A(N);
71    for(int &x:A)cin>>x;
72    MoRollBack mo(N,Q);
73    for(int i=0;i<Q;i++){
74        int l,r;
75        cin>>l>>r;
76        mo.add(l,r);
77    }
78    vector<int>cnt(1000001),ans(Q);
79    vector<int>hist;
80    int distinct=0,snap=0;
81    auto undo=[&]() {
82        int i=hist.back();
83        hist.pop_back();
84        if(--cnt[A[i]]==0)
85            distinct--;
86    };
87    auto add=[&](int i){ // iを追加
88        hist.push_back(i);
89    };
90    if(cnt[A[i]]++==0)
91        distinct++;
92    cout<<"add "<<i<<" -> ["
93        <<mo.l<<","<<mo.r<<")\n";
94    };
95    auto rem=[&](int q){ // q 番目の答えを記録
96        ans[q]=distinct;
97    };
98    cout<<"query "<<q<<" = ["
99        <<mo.l<<","<<mo.r<<")\n";
100    };
101    auto reset=[&]() { // 状態を空にする
102        while(!hist.empty())undo();
103        snap=0;
104    };
105    };
106 };

```

rollback_mo.cpp (cont.)

```

107 auto snapshot=[&]() { // 現在位置を保存
108     snap=hist.size();
109 };
110 auto rollback=[&]() { // snapshot まで巻き戻す
111     while((int)hist.size()>snap)
112         undo();
113 };
114 mo.run(add,rem,reset,snapshot,rollback);
115 for(int x:ans)
116     cout<<x<<"\n";
117 */

```

wavelet_matrix.cpp

```

1 template <class T,
2     int LOG = numeric_limits<
3         make_unsigned_t<T>>::
4         digits> //LOGは log_2 A_i より大きい値を
5     選択
6 struct
7 WaveletMatrix { //WaveletMatrix<int,10>
8     など
9     using U = make_unsigned_t<T>;
10    int n;
11    array<int, LOG> mid{};
12    array<vector<int>, LOG> s;
13    WaveletMatrix(const vector<T>& A)
14        : n(A.size()) { // 非負整数列 A から構築
15        vector<U> a(n), b(n);
16        for (int i = 0; i < n; i++) {
17            if constexpr (is_signed_v<T>)
18                assert(A[i] >= 0);
19            a[i] = (U)A[i];
20        }
21        for (int h = LOG - 1; h >= 0; h--) {
22            s[h].assign(n + 1, 0);
23            for (int i = 0; i < n; i++)
24                s[h][i + 1] =
25                    s[h][i] + (a[i] >> h & 1);
26            mid[h] = n - s[h][n];
27            int p = 0, q = mid[h];
28            for (U x : a)
29                ((x >> h & 1) ? b[q++] : b[p++]) =
30                    x;
31            a.swap(b);
32        }
33    T access(
34        int k) const { // 元配列の A[k] を返す
35        U x = 0;
36        for (int h = LOG - 1; h >= 0; h--) {
37            int o = s[h][k], b = s[h][k + 1] - o;
38            if (b)
39                x |= U(1) << h, k = mid[h] + o;
40            else
41                k -= o;
42        }
43        return (T)x;
44    }
45    int rank(T x, int l,
46        int r) const { // [l,r) にある x の個数
47        if constexpr (is_signed_v<T>)
48            if (x < 0)
49                return 0;
50        U y = (U)x;
51        if constexpr (LOG <
52            numeric_limits<U>::digits)
53            if (y >> LOG)

```

wavelet_matrix.cpp (cont.)

```

53        return 0;
54        for (int h = LOG - 1; h >= 0; h--) {
55            int L = s[h][l], R = s[h][r];
56            if (y >> h & 1)
57                l = mid[h] + L, r = mid[h] + R;
58            else
59                l -= L, r -= R;
60        }
61        return r - l;
62    }
63    int rank(T x,
64        int r) const { // [0,r) にある x の個数
65        return rank(x, 0, r);
66    }
67    T kth(int l, int r, int k)
68        const { // [l,r) の k 番目に小さい値 (
69        0-indexed)
70        U x = 0;
71        for (int h = LOG - 1; h >= 0; h--) {
72            int L = s[h][l], R = s[h][r],
73                z = r - l - (R - L);
74            if (k < z)
75                l -= L, r -= R;
76            else
77                x |= U(1) << h, k -= z,
78                l = mid[h] + L, r = mid[h] + R;
79        }
80        return (T)x;
81    }
82    T kth_largest(int l, int r,
83        int k) const { // [l,r) の k 番目に大きい値
84        return kth(l, r, r - l - 1 - k);
85    }
86    int less(int l, int r, T x)
87        const { // [l,r) にある x 未満の値の個数
88        if constexpr (is_signed_v<T>)
89            if (x <= 0)
90                return 0;
91        U y = (U)x;
92        if constexpr (LOG <
93            numeric_limits<U>::digits)
94            if (y >> LOG)
95                return r - l;
96        int c = 0;
97        for (int h = LOG - 1; h >= 0; h--) {
98            int L = s[h][l], R = s[h][r],
99                z = r - l - (R - L);
100            if (y >> h & 1)
101                c += z, l = mid[h] + L,
102                r = mid[h] + R;
103            else
104                l -= L, r -= R;
105        }
106        return c;
107    }
108    int range_freq(int l, int r, T lo, T hi)
109        const { // [l,r) で lo<=A[i]<hi の個数
110        return less(l, r, hi) - less(l, r, lo);
111    }
112    optional<T> prev_value(int l, int r,
113        T x) const { // [l,r) の x 未満の最大値
114        int k = less(l, r, x);
115        if (!k)
116            return nullopt;
117        return kth(l, r, k - 1);
118    }

```

wavelet_matrix.cpp (cont.)

```
118 optional<T> next_value(int l, int r,  
119     T x) const { // [l,r)のx以上の最小値  
120     int k = less(l, r, x);  
121     if (k == r - l)  
122         return nullopt;  
123     return kth(l, r, k);  
124 }  
125 };
```